

# Always-On, Always-Mine: Federated Recommendation Systems on Personal Home Routers

Zhengquan Li  
University of Michigan - Dearborn  
Dearborn, MI, USA  
zqli@umich.edu

Myungjin Lee  
Cisco Research  
San Jose, CA, USA  
myungjle@cisco.com

Zheng Song  
University of Michigan - Dearborn  
Dearborn, MI, USA  
zhesong@umich.edu

## Abstract

As concerns around end-user privacy continues to grow, Federated Recommendation Systems (FRS) have emerged as a privacy-preserving alternative to traditional cloud-based recommendation systems. They keep personal data “*always mine*”—never leaving the user’s device during both training and inference of the recommendation models. However, our empirical study shows that phone-based FRS often suffer from poor recommendation accuracy. This is because, in real-world settings, only a small fraction of phones can contribute data to model updates, as they are often not charging or connected to Wi-Fi when training occurs. In this paper, we propose using “*always-on*” personal home routers—devices that are continuously powered and consistently network-connected—as a promising alternative to intermittently available mobile phones for FRS. Despite its potential, router-based FRS introduces two key challenges. First, routers have more limited and variable computational resources than phones, leading to slower model convergence during federated training, as the system often waits on slower but data-rich devices. Second, routers introduce an additional layer of distributed infrastructure, increasing the complexity of managing training and inference workflows for application developers. To address these challenges, we present a middleware framework for router-based FRS that includes: (1) a customized federated training strategy that maximizes data utilization from slower routers without significantly increasing training time, and (2) a declarative programming model that simplifies the integration of router-based FRS into applications. Our evaluation shows that the router-based FRS improves recommendation accuracy by up to 53% compared to phone-based systems, while our framework reduces convergence time by up to 43% relative to state-of-the-art federated training methods, with minimal programming effort and runtime overhead.

## CCS Concepts

• General and reference → Performance.

## Keywords

Federated Learning, Recommendation Systems, Edge Computing

### ACM Reference Format:

Zhengquan Li, Myungjin Lee, and Zheng Song. 2025. Always-On, Always-Mine: Federated Recommendation Systems on Personal Home Routers. In *26th International Middleware Conference (MIDDLEWARE '25)*, December

15–19, 2025, Nashville, TN, USA. ACM, Nashville, TN, USA, 14 pages. <https://doi.org/10.1145/3721462.3770775>

## 1 Introduction

Recommendation systems (RS) have become a cornerstone of modern applications (apps) across domains such as e-commerce [26], social media [20], and entertainment [3], helping users discover content, products, and social connections aligned with their individual interests. These systems typically operate in a cloud-based manner, where user data—such as browsing histories and click patterns—is uploaded to centralized servers. This data is then used to train Machine Learning (ML)-based recommendation models and perform inference to generate personalized recommendations [45].

Recently, growing awareness of user privacy, along with the enforcement of regulations such as GDPR [41] in the EU, CCPA [47] in the USA, and PIPL [2] in China, have made it increasingly difficult—and in some cases infeasible—for app providers to collect and aggregate user data from personal devices for centralized model training [35]. To address these privacy concerns, *Federated Recommendation Systems (FRS)* [23, 51, 55] offer an improvement over cloud-based RS by keeping personal data “*always mine*”—never leaving the user’s device during training and inference.

At the core of FRS is federated learning (FL) [57], a distributed training paradigm that updates model parameters by training directly on users’ private data stored on their smartphones. Typically, a centralized FL server coordinates client devices through multiple rounds of training. In each round, a subset of devices is selected to receive the latest global model, perform local training on-device, and upload their model updates for aggregation. Because this process requires significant energy, computational resources, and network bandwidth, existing FL practices typically select only devices that are idle, connected to Wi-Fi and plugged into a power source during the training window [8, 54].

However, our empirical study (see Sec. 2) reveals that phone-based FRS suffers from low recommendation accuracy. While frequent training can help keep recommendation models up to date, each training window must remain brief to avoid interfering with users’ daily phone activities. During such short windows, many devices remain unavailable, as confirmed by real-world device status traces in our study. As a result, user data from these unavailable devices cannot be incorporated into training, ultimately degrading the quality of the learned recommendation model.

In this paper, we propose an FRS approach that leverages home routers as replacements for mobile phones in performing computationally intensive training and recommendation tasks. Like phones, home routers are fully user-controlled devices (*always-mine*), making them suitable for storing and processing private data in a privacy-preserving manner. Unlike phones, routers are



This work is licensed under a Creative Commons Attribution 4.0 International License. *MIDDLEWARE '25, Nashville, TN, USA*  
© 2025 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-1554-9/2025/12  
<https://doi.org/10.1145/3721462.3770775>

typically powered on 24/7 (*always-on*), maintain stable network connectivity, and often have spare computational and communication resources, making them well-suited for continuous learning tasks. By shifting federated training from phones to routers, our approach significantly improves training availability and, ultimately, recommendation accuracy, while preserving user privacy.

While router-based FRS offers compelling advantages, realizing this vision in practice introduces two key challenges (See Sec. 3): (C1) *Straggler mitigation without sacrificing accuracy*: Routers have high heterogeneous and resource-constrained characteristics due to their hardware limitations and varying traffic patterns. When existing straggler mitigation strategies are directly applied without accounting for the unique characteristics, some routers may be excluded from the short training window—even if they contain valuable user data critical for maintaining model accuracy.

(C2) *Reducing developer adoption complexity*: Traditional FL is already challenging to adopt, as developers must integrate FL logic into the app in addition to implementing its core functionality. Router-based FRS introduces another layer of complexity by involving routers in the FL workflow, necessitating coordination across the app runtime, routers, and cloud servers. This distributed setup demands substantial modifications to the app logic, creating a significant barrier to adoption.

To address these challenges, we develop a lightweight middleware system that: (1) schedules training to include slower yet valuable routers by selectively extending round deadlines and leveraging idle periods—maximizing user data inclusion; and (2) provides a declarative programming model that allows developers to integrate FRS using simple annotations, requiring minimal code changes. This design improves both model accuracy and training efficiency, while supporting practical, low-effort deployment. Using real-world resource usage traces from home routers, we evaluate two open-source recommendation models across four datasets. Our results show that the proposed solution improves recommendation accuracy by up to 53% compared to phone-based FRS and reduces model convergence time by up to 43% compared to state-of-the-art training scheduling strategies. We also evaluate our middleware on a testbed, demonstrating that it requires minimal programming effort from developers and imposes negligible overhead on app runtime.

In summary, this paper makes the following contributions:

- 1) We propose a novel router-based FRS architecture that overcomes the limitations of phone-based FRS, achieving both high recommendation accuracy and user privacy guarantees.
- 2) We design a middleware system to support router-based FRS, featuring: (a) a customized scheduling strategy that accelerates training on resource-constrained, heterogeneous routers to ensure timely recommendations; and (b) an annotation-based programming model that enables developers to easily integrate recommendation functionality into their apps.
- 3) We conduct extensive trace-driven emulation and testbed experiments to show the effectiveness and efficiency of our solution.

## 2 Background and Motivation

In this section, we introduce necessary background of FRS, and present our empirical study results to show the limitations of phone-based FRS, which motivates this work.

### 2.1 Recommendation Systems

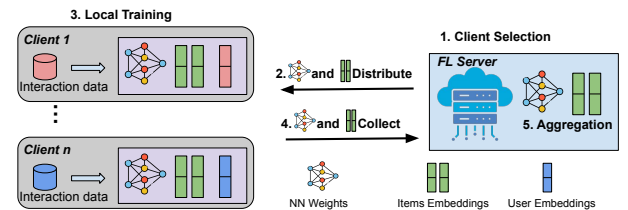
Recommendation systems (RS) help users discover relevant items—such as news, movies, or products—by learning from historical interactions. Modern RS represent users and items as embedding vectors in a shared low-dimensional space [7, 29]. A *user embedding* captures preferences based on past behavior, while an *item embedding* reflects attributes like category, content, or popularity. For instance, a user who often reads political news will be closer in vector space to articles on similar topics. The smaller the distance between user and item embeddings, the more likely the system is to recommend the item.

In most apps, users interact with items—such as watching a movie or clicking a product—through explicit feedback (e.g., ratings) or implicit signals (e.g., clicks or views). RS learn from this history by mapping user-item pairs to relevance scores using an interaction function that combines their embeddings. This function may be a simple dot product [29, 37] or a neural network that captures complex patterns [16, 22]. The model is trained to minimize the difference between predicted and actual feedback by adjusting user/item embeddings and model parameters, guided by a loss function suited to the feedback type.

Despite their widespread success, traditional recommendation systems are usually deployed on cloud servers, where they rely on centralized data collection. This introduces significant privacy risks. User-item interaction data can be highly sensitive and, if exposed, may allow malicious parties—or even cloud providers—to infer personal information, such as browsing habits, political views, or financial status. These concerns highlight the importance of designing recommendation systems that reduce privacy risks while still delivering personalized results.

### 2.2 Federated Recommendation Systems

Federated learning (FL) has emerged as a privacy-preserving machine learning paradigm [57]. It has been adapted to recommendation systems in the form of federated recommendation systems (FRS), which address the privacy risks of cloud-based RS by keeping both model training and inference entirely on user devices. In FRS, certain parts of the model—such as item embeddings and shared neural network parameters—are treated as public and are maintained by a central FL server. Meanwhile, each user retains their own interaction data and private user-specific embeddings locally on their device.



**Figure 1: Iterative Training Workflow of FRS. NN weights and items embeddings are public parameters, and user embeddings are private parameters.**

Figure 1 illustrates this training workflow. At the start of training, the FL server initializes the public model parameters, while each user device initializes its private embedding. In each training round, the server selects a subset of user devices and sends them the current

public model parameters. Each selected client then uses its local interaction data to compute updates based on its own data. These updates include both adjustments to the local user embeddings and gradients for the shared public model parameters. The user device applies updates to its private embeddings locally, while the gradients for public parameters are sent back to the server. The server aggregates updates from all participating clients and uses them to refine the global model. This process is repeated across multiple rounds until the model converges.

After training completes, the model remains on each client device to serve local inference requests. When a user initiates a recommendation query—for example, requesting a list of shopping items—the client device uses its private user embedding along with the updated global model to generate and return personalized recommendations.

Throughout both training and inference, user interaction data and user-specific embeddings remain on the client side. At no point does the FL server access historical interactions or make predictions on behalf of users, thereby preserving user privacy.

### 2.3 Limitations of Current Phone-based FRS

In conventional FL, data-generating devices serve as clients and interact directly with the FL server for distributed training [11]. In the context of FRS, these clients are typically users’ mobile phones. While phone-based FRS protects user privacy, we conduct an empirical study to show that it suffers from low recommendation accuracy in real word settings, which limits its practical adoption.

FL tasks involve local model training, which consumes phone computational resources and energy, as well as transmission of model parameters, which uses network bandwidth. To avoid slowing down phone performance, draining battery, or consuming users’ cellular data, current FL practices impose strict availability constraints—only allowing phones that are idle, charging, and connected to WiFi to participate in training [8, 54]. While these constraints eliminate negative impacts on users, they also significantly limit the pool of available devices for FL tasks. For instance, Google observed lower next-word prediction accuracy in the Gboard app during daytime hours, as only a small number of phones met these constraints—and those that did tended to represent a skewed user population [54]. With most phones available at night, the training window is typically set to 24 hours to update models.

While a 24-hour training window may be acceptable for some ML-based apps, it is problematic for apps that require *online learning*, particularly those that rely on recommendation services—such as news platforms or interactive social networks (e.g., Facebook, Twitter, LinkedIn). These apps generate large amounts of data with high temporality, which generally become obsolete in a matter of hours or even minutes [38]. Therefore, these recommendation models must be updated in short time period, so as to delivery fresh and relevant recommendation for high-quality user experience.

| Field         | Description                           | Example                   |
|---------------|---------------------------------------|---------------------------|
| user_id       | Anonymized user id.                   | xxxxxyyzzz                |
| device_model  | Device type                           | SM-A300M                  |
| screen_state  | Screen lock or unlock                 | screen_lock               |
| network_state | Network condition                     | 2G/3G/4G/5G/WiFi          |
| battery_state | Battery charging state, battery level | battery_charged_off 96.0% |
| time          | Time at current state                 | 2020-01-29 05:52:16       |

Table 1: An Entry Example in the Phone State Datasets [53].

However, within a short training time window—such as a few hours or even less—many phones are unable to participate in training due to strict availability constraints. To quantify this limitation, we analyzed a dataset of 136,000 smartphones that recorded state changes for each phone across multiple countries over the course of a week [53]. An example phone entry is shown in Table 1, which includes information such as screen status, network condition, and battery state. Specifically, we first processed the dataset to extract the available time slots for each phone—defined as periods when the screen state is locked (i.e., the phone is idle), the battery charging state is on, and network state is WiFi. For a given time window, a phone is considered available if it has at least one such time slot within that window. We then calculated the *ratio of unique phones that were available* relative to the total number of phones.

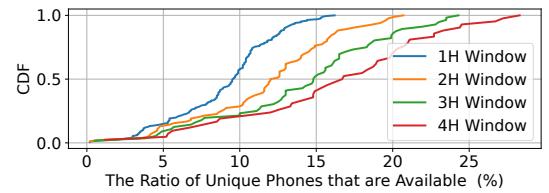


Figure 2: CDF of Phone Availability Ratio with Different Training Time Windows

Figure 2 presents the CDF of phone availability ratios under different time windows (1 hour, 2 hours, 3 hours, and 4 hours) over the week. We observe the following: 1) The overall availability is quite low. For example, under a 1-hour time window, the maximum availability ratio across the week reaches only about 17%. 2) Longer time windows lead to higher availability. Specifically, the maximum availability under a 4-hour window is approximately 10% higher than that under a 1-hour window.

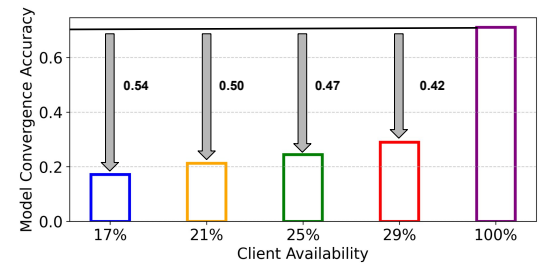


Figure 3: Client Availability vs. Model Convergence Accuracy

However, as discussed above, the training time window for recommendation models cannot be arbitrarily long due to the volatile nature of user data. In such cases, a significant portion of user data cannot be utilized for model updates, leading to degraded recommendation quality. To quantify this impact, we selected an open-source federated recommendation model, PFedRec [56], (details of the model and experimental configurations are provided in Sec. 5) and trained it using the maximum availability ratios corresponding to four different time windows. We then compared the resulting model convergence accuracy with an oracle case in which all phones are assumed to be available.

As shown in Figure 3, we observe that: 1) Higher phone availability leads to higher model accuracy, as more unique user data



is incorporated into the training process. 2) The accuracy gap between the constrained cases (17–29% availability) and the oracle case (100% availability) is substantial. For example, at 17% availability, the model suffers nearly a 54% reduction in accuracy compared to the oracle scenario.

This drastic decline occurs because a large portion of unique user data is excluded from training—never even used in a single training round—since the corresponding phones do not meet the availability requirements. Unlike other FL tasks (e.g., computer vision, LLM) where training windows span many hours to several days [1]—allowing most phones to eventually participate—recommendation tasks operate within much shorter windows. As a result, many phones never become available in time, causing the final model to be trained on substantially less unique user data and thus resulting in lower accuracy.

#### Observation 1

Modern apps require recommendation models to be trained within short windows to deliver timely, high-quality user experiences. However, most user phones cannot participate due to strict FL availability constraints (i.e., idle, charging, and connected to WiFi). As a result, much user data is excluded from training, leading to lower model accuracy.

## 2.4 Consistent Availability of Home Routers

The goal of enforcing availability constraints on phones is to eliminate any negative impact on users. Among the three constraints, being idle, charging, and connected to WiFi, the latter two directly correspond to a home router's constant power supply and Broadband network connectivity, which are inherently satisfied. The "idle" constraint, however, requires further interpretation in the context of routers.

For phones, enforcing training only when the device is idle ensures that user-facing tasks are not affected by resource contention from training tasks. In contrast, home routers operate differently. These routers run Linux-based operating systems (OS) such as OpenWRT [12] and Tomato [14], where core networking functionalities—such as packet forwarding—are managed by the Linux networking subsystem in kernel space. This separation means that additional tasks, such as model training, can be executed in user space without interfering with the router's primary networking responsibilities. Even in the presence of resource contention, the OS inherently prioritizes kernel-space processes, ensuring that training does not degrade network performance [15].

Based on this observation, we regard home routers as being consistently available for model training. Next, we present empirical measurements to further confirm both the availability of spare resources on routers in typical home environments and the zero impact of running user-space tasks on their network performance.



Figure 4: Testbed Configuration

**Underutilization of Resources** We analyzed the resource usage of home routers under typical network traffic conditions. Using pre-captured smart home traffic from the ACI Network Traffic Dataset 2023 [6], we replayed two days of benign traffic, representing real-life scenarios, to a home router and monitored CPU and memory usage. The dataset includes traffic from various devices such as mobile phones, PCs, and IoT devices (e.g., smart TVs, bulbs, and Amazon Voice Assistants) over five days, with both malicious and benign traffic.

To replay the traffic, we set up a testbed (Figure 4) consisting of an off-the-shelf router (GL-MT1300 powered by an MT7621A CPU @ 880MHz with 256MB memory), a Raspberry Pi (Model 4B ARMv7 with 4 CPUs and 2GB RAM) as a client connected to the router's LAN port, and another Raspberry Pi as a server connected to the WAN port. Traffic replay was performed using Tcpreplay, setting source and destination IP/MAC addresses to match the client and server. Figure 5 illustrates the router's resource usage measured every second. Both CPU and memory were underutilized, with CPU usage staying below 50% and memory usage under 20% of total capacity (256MB). Even during peak traffic loads, sufficient spare resources were available.

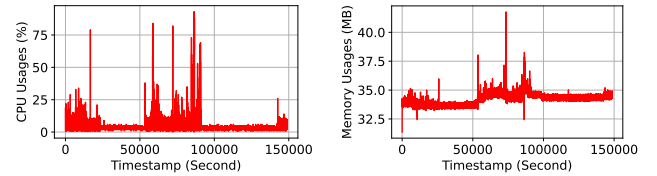
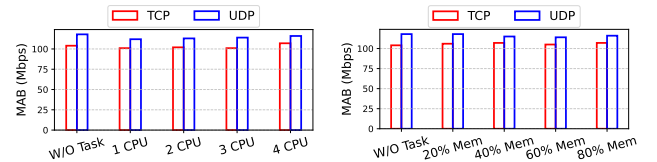


Figure 5: Resources Usages on Home Router

**Full Routing Performance During Additional FL Tasks** Since network traffic processing and running extra tasks, such as FL training, can compete for system resources (e.g., CPU, memory) on the router, we compare the router's network performance while replaying the home network traffic under two scenarios: 1) *No Extra Task*—As the baseline scenario, no extra tasks run on the router; 2) *Extra Tasks with Varying Workloads*—A Python program is executed on the router to occupy different CPU cores and varying proportions of memory. To stress a CPU core, an infinite loop is run in a single thread. To control memory usage, a list of varying lengths is allocated. The program runs in the user space of the router OS with a default priority of 0 and serves as an emulation of FL model training and inference.



(a) Different CPU Workloads

(b) Different Mem. Workloads

Figure 6: Network Perf. of Router under Different Workloads

We measured the router's maximum achievable bandwidth (MAB) with Iperf3, a network measurement tool that calculates bandwidth by sending traffic from the client to the server. As shown in Figure 6, the router's network performance remains unaffected, even when all 4 CPU cores are fully utilized and 80% of memory is consumed, indicating no noticeable degradation. This result

confirms that the router OS prioritizes resources for critical kernel-space functions—such as packet forwarding—over user-space tasks like model training, thereby ensuring that the router’s primary functionality remains unaffected during resource contention.

#### Observation 2

Home routers offer consistent spare resources beyond their primary role of packet forwarding. We leverage them for federated recommendation training, enabling broader user data inclusion and significantly improving model accuracy.

### 3 Challenges and Solution Overview

While we confirm that routers are consistently available for recommendation model training, applying this router-based FRS in real world still faces the following challenges:

*C1: How to efficiently conduct federated training on resource-limited and heterogeneous home routers?* In federated training, devices often have heterogeneous computational capabilities. Some become **stragglers**, taking significantly longer to complete local training than others. These stragglers create a bottleneck in each training round, as faster devices must wait for slower ones to finish, ultimately slowing down model convergence. However, it’s still important to include them, as their data could be valuable for the final model. Using time-to-accuracy as a metric and aiming to optimize this metric, state-of-the-art federated training systems such as Oort [30], PyramidFL [33], REFL [1], and FLOAT [27] mitigate this issue using a common principle: selecting devices with high overall utility—balancing both data utility (i.e., contribution to model accuracy) and system utility (i.e., ability to complete training within a specified deadline)—in each round.

We use Oort as an example to illustrate the utility function used for device selection:

$$\text{Util}(i) = \underbrace{|B_i| \sqrt{\frac{1}{|B_i|} \sum_{k \in B_i} \text{Loss}(k)^2}}_{\text{Data utility}} \times \underbrace{\left(\frac{T}{t_i}\right)^{1(T < t_i) \times \alpha}}_{\text{System utility}} \quad (1)$$

For data utility,  $B_i$  represents the set of local data samples on device  $i$ , and  $\text{Loss}(k)$  denotes the training loss for sample  $k$ . For system utility,  $T$  is the developer-specified target deadline for each round,  $t_i$  is the wall-clock time taken by client  $i$  to complete training,  $\alpha$  is a penalty factor, and  $1$  is an indicator function. If the client finishes training within the target deadline ( $t_i \leq T$ ), its system utility remains 1. Otherwise, the utility is penalized, reducing the client’s likelihood of being selected due to slower training speed.

In router-based FRS, the straggler problem becomes even more pronounced if existing strategies are applied directly. Although routers are generally consistent, their available resources can fluctuate significantly due to spikes in network traffic, which directly impacts their system utility. Routers with low hardware configuration and/or experiencing high traffic could be excluded from training due to their low system utility because their training time can far exceed the developer-preferred round deadline ( $t_i \gg T$ ). However, this contradicts the core goal of router-based FRS: leveraging a broader set of user data to improve recommendation accuracy. Excluding slower routers from training reduces data diversity. This

issue is further exacerbated by the fact that a single router may contain multiple users; omitting one router from training can result in the loss of multiple users’ data, amplifying the negative impact on model accuracy.

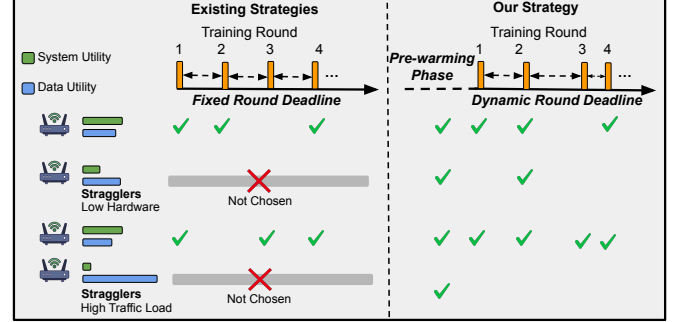


Figure 7: Existing Training Strategies vs. Our Strategy

**Solution:** To mitigate the straggler issue without entirely discarding the data contributed by slower routers, we introduce two key mechanisms into the federated training process:

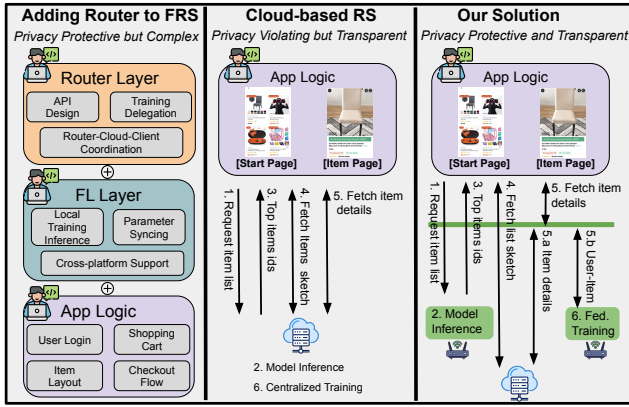
- (1) *Dynamic Round Deadlines:* Instead of enforcing a fixed target training deadline across all rounds, we dynamically adjust deadlines based on each router’s system utility and data value. Slower routers with high-value data are selectively included by allowing slightly longer deadlines in specific rounds. While this increases the duration of some rounds, it improves the ratio between data utility and time, avoids excluding important data, and improves overall model convergence speed.
- (2) *Pre-warming Phase:* To ensure that each router’s user data is utilized at least once, local training is performed during the time gap between consecutive training windows. In this phase, routers train on accumulated user data regardless of their system utility. This approach not only guarantees initial data contribution but also helps the system profile each router’s utilities for round selection in the future online training window.

Together, these mechanisms mitigate the negative impact of straggler routers while preserving contributions from a broad range of user data, ensuring that the model converges to high accuracy within a short time window. As demonstrated in Figure 7, fixed-deadline strategies exclude training data from the 2nd and 4th routers due to their low system utility, even though their data utility is high. In contrast, our pre-warming strategy ensures that data from the 4th router is included at least once and incorporates the 2nd router in a training round with slightly extended deadline to leverage its higher data utility.

*C2: How can developers easily integrate router-based FRS into their apps?* FRS is built on top of traditional FL, which is inherently complex and difficult to adopt in practice. As shown in Figure 8, developers must handle many responsibilities beyond basic app logic—such as integrating on-device training and inference using platform-specific FL SDKs (e.g., Android, iOS), and managing model parameter synchronization with a central FL server. Introducing routers into FRS adds yet another layer of complexity: developers now need to understand how to interact with router infrastructure (e.g., designing custom APIs), manage app data flows through

routers, and coordinate training and inference across the app, the router, and the cloud. This significantly increases the development burden and hinders the practical adoption of router-based FRS.

In contrast, cloud-based RS are much easier for developers to adopt. All recommendation functionalities—both model training and inference—are fully handled on the cloud side, with no need for developers to manage infrastructure or training logic. For example, in the shopping app *Wish*, when a user opens the *Start Page*, the app sends a recommendation request to the cloud server (1). The server performs model inference (2) and returns a ranked list of item IDs (3). The app then uses these IDs to retrieve item metadata, such as titles and thumbnails, from the cloud (4). If the user clicks on an item, another request fetches detailed information such as images, descriptions, and ratings (5). All user–item interactions are automatically logged in the cloud and later used to update the model (6). From the developer’s perspective, the entire recommendation pipeline operates transparently behind the scenes.



**Figure 8: Traditional Programming Approaches vs. Our Middleware When Integrating Recommendation Functionality**

**Solution:** To provide integration transparency like cloud-based RS, we design the router-based FRS as a middleware system positioned between the app runtime and the cloud server. It provides a dedicated programming interface, enabling developers to simply add text annotation of modern languages such as Java and Python to the app’s source code to mark the HTTP requests of the cloud-based RS. At runtime, the app-side middleware automatically redirects these annotated requests to either the router-side middleware or the cloud server, as required, to fetch recommended items and the necessary data for the app.

The workflow proceeds as follows: When the *Start Page* is opened, the annotated recommendation request is intercepted and forwarded to the router (1). The router performs local model inference and generates a ranked list of item IDs (2), which it then returns to the app (3). The app uses these IDs to retrieve item metadata (e.g., titles, thumbnails) from the cloud (4). When a user clicks on an item, the app sends a request to the cloud for detailed content (5a), while the middleware simultaneously logs the user–item interaction to the router (5b). The router stores these interactions and uses them in future federated training rounds with the parameter server (6).

By offloading complexity to the middleware and preserving the cloud-based development interface, our system ensures that

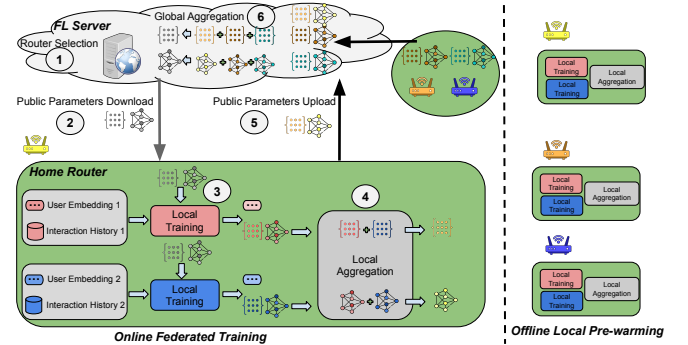
router-based FRS can be integrated with minimal programming effort—providing cloud-like transparency even with the added layer of router-based execution.

## 4 Middleware System Design

In this section, we present the design details of our middleware system, which includes a federated training strategy to ensure high model accuracy within a short time window, and a programming model that facilitates transparent integration of recommendation functionality into real-world apps.

### 4.1 Federated Model Training on Home Routers

Our training setup consists of a centralized FL server and a distributed set of home routers. Each router locally stores user interaction histories and collaborates with the server to train a recommendation model for each app. The high-level design of this workflow is illustrated in Figure 9, which includes two stages: online federated training and offline local pre-warming.



**Figure 9: Training Workflow of Router-based FRS**

#### 4.1.1 Online Synchronous Federated Training.

Online training occurs when the FL server actively coordinates routers to update the model within a designated time window. Unlike phone-based FL training, router-based FRS requires a specialized workflow that addresses its unique characteristics—such as each router potentially hosting data from multiple users, and the separation of model parameters into private and public components. Our customized workflow consists of the following six steps:

- (1) *Router Selection*: At the beginning of each training round, the FL server applies a dynamic round deadline strategy to select a subset of routers to participate in training and calculate the training deadline.
- (2) *Public Parameters Download*: Each selected router downloads the public model parameters from the server, including item embedding vectors and the weights of the neural network.
- (3) *Local Training for Individual Users*: Each router performs local training using the interaction histories of its users. This training updates each user’s private embedding, as well as a local copy of the item embeddings and neural network weights. Since user embeddings are personalized representations of individual preferences, the training is conducted separately for each user if multiple users of the same app are present on the router. Due to limited computational resources, these user-specific training processes are executed sequentially on the router.

- (4) *Local Aggregation*: Once the router completes local training for all of its users, it retains a separately updated embedding for each user. In addition, it accumulates multiple updated versions of the item embeddings and neural network weights—one from each user’s training process. The router then performs an internal aggregation of these user-level updates to produce a single combined update for the item embeddings and another for the neural network weights. This aggregation step reduces communication overhead by minimizing the volume of data sent to the server.
- (5) *Public Parameters Upload*: The router uploads its locally aggregated updates for both the item embeddings and the neural network weights to the server.
- (6) *Global Aggregation*: After receiving updates from all selected routers, the server aggregates them to refine the global model parameters, including both the item embeddings and the neural network weights.

This entire process repeats over multiple rounds until the model converges or its accuracy meets a predefined threshold.

**Router Selection with Dynamic Deadline:** As previously discussed, we propose a novel router selection mechanism with a dynamic round deadline. The key idea is to relax the training deadline when including slower routers leads to a higher overall utility per unit time. To achieve this, we generate multiple candidate deadlines (e.g., 1.5×, 2.0× the original deadline). For each candidate, we identify the subset of routers that can complete training within the deadline, compute the utility of the top  $K$  routers (i.e., the number selected per round), and choose the deadline that maximizes the utility-to-time ratio.

---

**Algorithm 1** Router Selection with Dynamic Round Deadline

---

```

1: Input: Routers  $\mathcal{R} = \{r_1, r_2, \dots, r_m\}$  with statistical utility  $U_{\text{stat}}(r)$  and training time  $t(r)$ ;
   Number of routers to select  $K$ ; Penalty factor  $\alpha$ 
2: Initialize: Optimal deadline  $T^* \leftarrow 0$ , max_utility  $\leftarrow -\infty$ 
3: Generate candidate deadlines  $\mathcal{T} \leftarrow \{t(r) \mid r \in \mathcal{R}\}$ 
4: for  $T \in \mathcal{T}$  do
5:   Initialize: utilities  $\leftarrow []$ 
6:   for  $r \in \mathcal{R}$  do
7:      $U(r) \leftarrow U_{\text{data}}(r) \cdot \left(\frac{T}{t(r)}\right)^{1(T < t(r)) \cdot \alpha}$ 
8:     Append  $(r, U(r))$  to utilities
9:   end for
10:  Sort utilities in descending order by  $U(r)$ 
11:  Select the top  $K$  routers  $\mathcal{R}_K$  from utilities
12:  Compute accumulated_utility  $\leftarrow \sum_{r \in \mathcal{R}_K} U(r)$ 
13:  Compute utility_to_time  $\leftarrow \text{accumulated\_utility} / T$ 
14:  if utility_to_time > max_utility_to_time then
15:    max_utility_to_time  $\leftarrow$  utility_to_time
16:     $T^* \leftarrow T$ 
17:    Save  $\mathcal{R}_K$  as the top  $K$  routers
18:  end if
19: end for
20: Output: Top  $K$  selected routers  $\mathcal{R}_K$ 

```

---

Algorithm 1 shows the selection process. At the beginning of each training round, the FL server has access to up-to-date profiling information for each router, including its data utility (based on prior training contribution) and its most recent training time. These are used to dynamically assess the router’s utility under different deadline configurations. The set of candidate deadlines is initialized using the observed training times of all available routers (Line 3), ensuring that the selection process considers both fast and slow routers. For each candidate deadline, the server first calculates the utility of every router by combining its data utility with a system

utility penalty, which reduces the overall score for routers expected to exceed the deadline (Lines 6–9). Routers with higher data value and reasonable training times are favored in this step. The server then ranks the routers based on their computed utilities and selects the top  $K$  routers under that specific deadline (Lines 10–11). The accumulated utility—the sum of utilities of the selected routers,  $\sum_{i \in \mathcal{R}_K} \text{Util}(i)$ —is recorded for each deadline candidate (Line 12).

After evaluating all candidate deadlines, the algorithm selects the one that yields the highest utility-to-time ratio as the optimal deadline for the current round (Lines 13–15). The corresponding top  $K$  routers under this optimal deadline are then chosen to participate in training (Line 16). This dynamic adjustment allows us to selectively include slower routers with high-value data by modestly increasing the round duration of certain rounds when beneficial, ultimately reducing overall convergence time and improving model accuracy via preserving the data diversity.

#### 4.1.2 Offline Asynchronous Local Pre-warming.

While dynamically adjusting the training deadline in each round improves the inclusion of user data and contributes to overall model accuracy, it still cannot guarantee participation from every user—particularly those hosted on extremely slow routers. However, ensuring that each user’s data is incorporated is critical, as the model must train a private embedding for each user to enable personalized recommendations.

To address this limitation, we introduce an offline, asynchronous pre-warming phase to ensure that each user’s data is utilized at least once. Since online training is periodically relaunched with newly generated interaction data to keep recommendations up to date, each router retains a copy of the trained model parameters after the completion of each online training session. Between consecutive online training windows, the router continuously accumulates user interaction data and trains the model locally whenever sufficient data is gathered for a training batch. During this offline phase, each router updates the model parameters independently—also logging metadata such as loss and training time—without requiring server synchronization. When a new online training window begins, each router sends its updated parameters to the server for aggregation, along with metadata to facilitate the router selection.

## 4.2 Programming Model and Runtime Support

Our programming model aims to handle the interactions between the app, home router, and cloud server, ensuring recommendation transparency similar to cloud-based RS. Based on the observations: 1) recommendation functionality are related to the requests of *Start Page* and *Item Page* and 2) mobile apps commonly use open-source HTTP client libraries (e.g., OkHttp, Apache HttpClient, or Retrofit) to issue HTTP requests, we can modify the HTTP client library to intercept these requests and conduct further processing accordingly. **Annotation-based Programming Interface:** We provide two Java annotations for app developers to mark HTTP requests of cloud-based RS. Specifically, `START_FETCH` is used for the start page request, and `ITEM_FETCH` is used for the item page request, as illustrated in Figure 10. The `id` field in both annotations uniquely identifies the HTTP requests by representing their base URLs without query parameters.



```

1  @Retention(RetentionPolicy.RUNTIME)
2  @Target(ElementType.FIELD)
3  public @interface START_FETCH {
4      String id();
5      String itemIDs_format();
6
7  @Retention(RetentionPolicy.RUNTIME)
8  @Target(ElementType.FIELD)
9  public @interface ITEM_FETCH {
10     String id();

```

Figure 10: Available Java Annotations

As shown in the middleware workflow in Figure 8, the *Start Page* fetches a list of recommended items from the router. For this to work seamlessly, the router must know how to organize the response data in a format that the app can parse—since the original app source code is designed to handle responses from the cloud. To support this, we introduce an `itemIDs_format` field in the `START_FETCH` annotation. This field is a JSON Schema [52] specified by the developer, which describes the expected format of the response containing recommended item IDs and how the app parses it. As illustrated in Figure 11, using the response of item IDs in the *Wish* shopping app as an example, it defines an array of objects (i.e., items in our case), where each object must contain a required "itemID" field. The "itemID" must be a 5-character lowercase alphanumeric string, following the regex pattern `^[a-z0-9]{5}$`. Any object missing the "itemID" field or not matching the pattern will be considered invalid. In contrast, the *Item Page* still fetches item detail data directly from the cloud, so there are no formatting compatibility issues.

```

1  {"type": "array",
2   "items": {
3     "type": "object",
4     "properties": {
5       "itemID": {
6         "type": "string",
7         "pattern": "^[a-z0-9]{5}$"
8       }
9     },
10    "required": ["itemID"]
11  }}

```

Figure 11: Example of a JSON Schema for `itemIDs_format`

```

1  public class StartPage {
2      private static final String BASE_URL = "http://www.wish.com/getList?";
3      private OkHttpClient client = new OkHttpClient();
4
5      // Fetching a list of recommended items via a HTTP request
6      @START_FETCH(id="http://www.wish.com/getList?",
7                  itemIDs_format="src/main/assets/itemIDs_schema.json")
8      public ItemList(String userID, Integer n) {
9          //userID will be shared only with router not with cloud
10         String url = BASE_URL + "userID=" + userID + "&n=" + n;
11         Request request = new Request.Builder().url(url).build();
12         Call call = client.newCall(request);
13         Response response = call.execute();
14     }
15 }
16
17 public class ItemPage {
18     private static final String BASE_URL = "http://www.wish.com/getItem?";
19     private OkHttpClient client = new OkHttpClient();
20
21     // Fetching a item's details via a HTTP request
22     @ITEM_FETCH(id="http://www.wish.com/getItem?")
23     public ItemDetail(String userID, String itemID) {
24
25         String url = BASE_URL + "userID=" + userID + "&itemID=" + itemID;
26         Request request = new Request.Builder().url(url).build();
27         Call call = client.newCall(request);
28         Response response = call.execute();
29     }
30 }

```

Figure 12: Example of Using Our Programming Interface

App developers add these annotations directly into the app's source code. As a concrete example, Figure 12 illustrates how a

developer annotates the classes representing the *Start Page* and *Item Page*. Specifically, for the `id` field, the developers explicitly defines its value in the source code, while for `itemIDs_format`, they specify the path to the JSON schema file.

**App Runtime Support:** To retrieve developer-added annotations, we utilize Reflection, a feature in Java language enables a program to inspect its own classes, interfaces, methods, fields, and annotations at runtime [34]. Specifically, when the app is run for the first time on a user's device (e.g., after a new installation or version update), the annotation retrieval module is triggered. This module uses Reflection to scan the app's source code and retrieve the two defined annotations (`START_FETCH` and `ITEM_FETCH`). The retrieved annotations are then stored within the modified HTTP client library to avoid repeated retrieval overhead during runtime.

Subsequently, the modified HTTP client library intercepts each outgoing HTTP request from the app to verify if its base URL matches any of the annotated requests (i.e., comparing `id` in annotations). The library then handles the interactions between the app runtime and other entities (i.e., router and cloud) as follows:

- For the request of the *Start Page*, the library parses the JSON schema specified by `itemIDs_format`, packages its content with the original request, and forwards it to the router runtime for processing. Specifically, the URL is modified by appending `itemIDs_format` as an additional query parameter, along with `userID` and `n`, as shown in Line 9 of Figure 12.
- For the request of the *Item Page*, the library performs two tasks: i) Interaction Data Reporting: It extracts the `userID` and `itemID` from the URL and sends this data to the router runtime as interaction data for future model training; ii) Privacy Preservation: It removes personally identifiable information (e.g., `userID`) from the URL before redirecting the request to the cloud server. This ensures that the user's interaction history remains on the router, preventing adversaries from learning it through the cloud server or the communication path between the router and the cloud.

**Router Runtime Support** The main responsibilities of our router runtime are: 1) serving the *Start Page* by generating recommended items through model inference, and 2) performing federated model training on received interaction data, as detailed in Sec. 4.1.

For each app, once the recommendation model is trained, all parameters are stored locally on the router. These include private user embeddings (potentially multiple, if the router serves multiple users of the same app), as well as public parameters such as item embeddings and neural network weights. When the router runtime receives a request from the *Start Page*, it first updates the list of non-interacted items based on the user's interaction history. Using the `userID` extracted from the request URL, the router retrieves the corresponding user embedding. It then performs model inference using this embedding along with the public parameters to generate relevance scores for each non-interacted item. Finally, it selects the top-`n` ranked items and constructs a response containing their `itemIDs` in the format specified by `itemIDs_format`, both of which (`n` and `itemIDs_format`) are extracted from the URL.



## 5 Evaluation

This section presents our evaluation of the middleware’s performance. We structure the evaluation around the following key questions, along with a summary of the corresponding findings:

**Q1:** How does router-based FRS perform compared to phone-based FRS?

*Findings:* Router-based FRS consistently outperforms phone-based FRS across all four datasets, achieving an average model accuracy improvement of 26% and a maximum improvement of 53%.

**Q2:** How effective is our federated training strategy compared to state-of-the-art straggler mitigation techniques?

*Findings:* Our strategy reduces convergence time by an average of 37% compared to Oort and FLOAT, without sacrificing accuracy.

**Q3:** How does our programming model impact developer effort and application performance?

*Findings:* Our programming model significantly reduces developer effort by eliminating the need to modify the app’s core logic, while introducing only negligible latency overhead at runtime.

### 5.1 Evaluation of Federated Model Training

We first evaluate federated model training performance by comparing our router-based FRS with phone-based FRS and assessing the effectiveness of our customized training strategy.

**5.1.1 Experimental Methodology.** We outline our experimental setup, including baseline approaches, evaluation metrics, selected recommendation models and datasets, and the construction of our emulation platform for router-based federated training.

**Evaluated Approaches** As the cloud-based RS inevitably cause privacy concern, we only evaluated the approaches that implement FRS model training:

1) *Phone-based FRS:* Mobile phones serve as FL clients for training. Each phone is randomly assigned a status trace from the dataset described in Sec. 2.3. At the beginning of each round, only phones that are idle, connected to WiFi, and charging check in with the FL server. Due to the limited availability of eligible phones as shown in our empirical study (Sec. 2.3), no additional client selection is performed; all available phones participate in training during each round during an online training window.

2) *Router-based FRS + Oort/FLOAT:* Home routers are used as FL clients for training. Unlike phone-based FRS, where devices may be intermittently unavailable, home routers are assumed to be consistently available throughout the online training window. In each round, the cloud server employs a straggler mitigation strategy—either Oort or FLOAT—to select a subset of routers for participation. Oort is chosen for its pioneering approach to client selection, which balances data utility and system utility. FLOAT is included as a more recent strategy that builds on the same trade-off principle, but introduces optimizations such as partial training to further mitigate the impact of stragglers.

3) *Router-based FRS + Our Training Strategy:* This is our proposed approach. Like the previous setup, it uses home routers as FL clients. In addition, it incorporates two custom mechanisms—dynamic round deadlines and offline pre-warming—on top of Oort or FLOAT to further optimize router selection and improve model convergence.

#### Evaluation Metrics

- **Final Model Accuracy:** This metric refers to the model’s accuracy on the test dataset after completing an online training window. Since routers have lower computational capacity than phones, we set the duration of the training window to the longest time required by any router-based FRS solution to reach convergence. This ensures that all solutions have sufficient time to converge—that is, to reach the point where the model’s test accuracy no longer improves.

In the recommendation domain, model accuracy is often evaluated using Hit Ratio  $HR@K$ , which measures the proportion of users whose ground-truth item appears in the top- $K$  recommendations. Formally, for each user  $u_i$ , let  $I_u$  denote the ground-truth item and  $\text{Rec}_u^K$  the top- $K$  items generated by the model. Then,  $HR@K$  is defined as:

$$HR@K = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathbb{I}(I_u \in \text{Rec}_u^K), \quad (2)$$

where  $\mathcal{U}$  represents the set of all app users, and  $\mathbb{I}(\cdot)$  is an indicator function that equals 1 if  $I_u \in \text{Rec}_u^K$ , and 0 otherwise.

- **Model Convergence Time:** This metric captures training efficiency by measuring the wall-clock time required for the model to reach convergence in the online training window.

We repeat each experiment three times with different random seeds and report the average values for these metrics.

**Datasets and Models** We evaluated two open-source federated recommendation models: *PFedRec* [56] and *FedIS* [18]. Both models are general-purpose recommendation systems that can be applied across various application scenarios. To ensure a diverse evaluation, we selected four datasets, summarized in Table 2. Specifically: *MovieLens-100K* [21]: Collected through the MovieLens website, this dataset contains movie ratings where each user has rated at least 20 movies. *Amazon-Video* [39]: Gathered from the Amazon site, this dataset includes product reviews and associated metadata. *Lastfm-2K* [9]: A music recommendation dataset where each user has a list of favorite artists and corresponding tags. *Steam* [48]: This dataset contains reviews and game-related information collected from the Steam PC gaming platform.

| Model   | Dataset        | #User  | #Item  | #Interaction |
|---------|----------------|--------|--------|--------------|
| PFedRec | MovieLens-100K | 943    | 1,682  | 100,000      |
|         | Amazon-video   | 63,836 | 8,072  | 11,830       |
| FedIS   | Lastfm-2K      | 1,892  | 17,632 | 92,834       |
|         | Steam          | 3,761  | 5,515  | 115,852      |

**Table 2: Evaluated Recommendation Models and Datasets**

The training and testing sets follow the configurations provided in the original open-source code. We trained *PFedRec* from scratch on the *MovieLens-100K* and *Amazon-Video* datasets. To simulate real-world scenarios where companies initially train a global model on a large, general dataset and then fine-tune it using personal data from their app users, we fine-tuned *FedIS*, a pre-trained model, using the *Lastfm-2K* and *Steam* datasets.

**User-to-Client Mapping** For phone-based FRS, each user in the dataset corresponds to a mobile phone acting as an FL client. For router-based FRS, we first determined how many users are assigned to each router, sampled from a uniform distribution between 1 and 5. We then randomly selected that many users from the dataset for each router while making sure no user is assigned to more than one router.

**Federated Training Emulation** The large number of users in the datasets requires hundreds and even thousands physical devices (i.e., phones and routers) to be FL clients to conduct the federated training. Due to the difficulties of accessing this large number of devices, we resort to build a training emulation platform to evaluate the training approaches. Our emulation platform consists of one Raspberry Pi (Model 4B ARMv7 with 4 CPUs and 2GB RAM) and one desktop PC (Intel i7-4790, 16GB Memory). The Raspberry Pi emulates virtual phones or routers, and the desktop PC that is 6 hop away acts as the FL server, handling client selection and aggregating model parameters. The two devices share a user-to-client mapping for each dataset. The on-device local training and the parameter aggregation are implemented by PyTorch [13].

At the start of each training round, the PC sends the chosen client IDs to the Raspberry Pi. In phone-based FRS, the chosen virtual phones use all Pi's resources to train locally one after another. Because only idle phones are allowed to conduct local training, there is no interference from other tasks. In router-based FRS, the selected virtual routers also train locally in sequence, but they must simultaneously handle network traffic. To simulate this interference, we randomly sample a continuous sequence of CPU usage data from the trace in Sec. 2.4 for each virtual router. During local training, the *stress-ng* [28] tool is used to occupy the designated percentage of CPU in the trace at the highest priority, creating realistic contention for system resources.

The wall-clock time for each training round is the maximum time taken by any selected virtual device. We only consider computation time (i.e., local model training on the Raspberry Pi), as prior works have shown that local computation time is typically the bottleneck in FL rounds, compared to communication overhead [43, 50, 53].

**Parameters Settings** Unless otherwise stated, all experiments are conducted using the following settings. The ratio of client selection in each round for router-based FRS is set to 20%. To accommodate the limited memory resources on real routers and avoid out-of-memory errors during local training, the mini-batch size for each recommendation model is set to 4. For algorithmic parameters, our approach, Oort and FLOAT use the default configurations provided in their GitHub repositories.

**5.1.2 Performance Comparison.** Figure 13 illustrates how model accuracy evolves over time across all four datasets.

**Final Model Accuracy** We compared the final model accuracy achieved within the training window, with durations determined as discussed earlier: 10,000 seconds for *PFedRec + MovieLens-100K*, 6,000 seconds for *PFedRec + Amazon-Video*, and 2,000 seconds for both *FedIS + Lastfm-2K* and *FedIS + Steam*. Our key observations are as follows:

- 1) Compared to phone-based FRS, router-based FRS consistently achieved higher final model accuracy across all models and datasets, providing an average 26.5% boost. This improvement arises because many phones did not participate during the training window due to the availability constraints, leaving the corresponding users' local model parameters untrained with their own data. Since the final model accuracy are evaluated on all users' test data, those untrained parameters lead to lower recommendation accuracy.
- 2) Router-based FRS showed less degree of accuracy improvements on *FedIS* over *PFedRec* compared to phone-based FRS : 12% for

*FedIS* versus 43% for *PFedRec*. Since *FedIS* is already pre-trained, even if some users do not participate, their local parameters still retain predictive ability. In contrast, *PFedRec* is trained from scratch, making the absence of user data more detrimental. Nonetheless, a 12% accuracy increase for *FedIS* remains significant, particularly in large-scale apps where even a modest improvement in accuracy can greatly enhance the experience of a vast number of users.

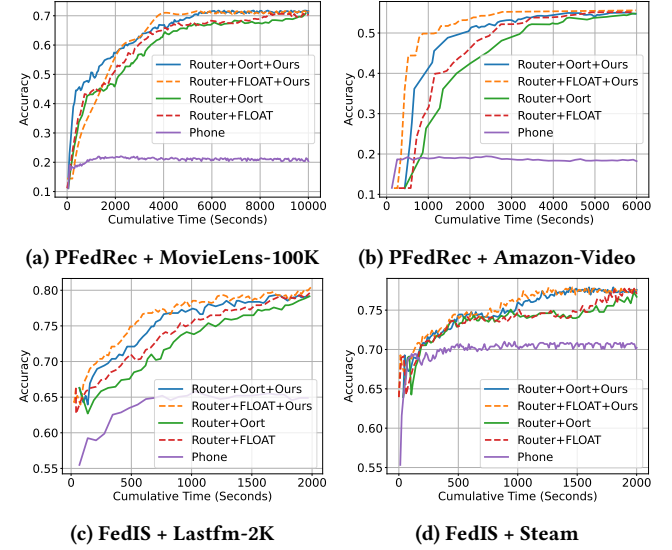
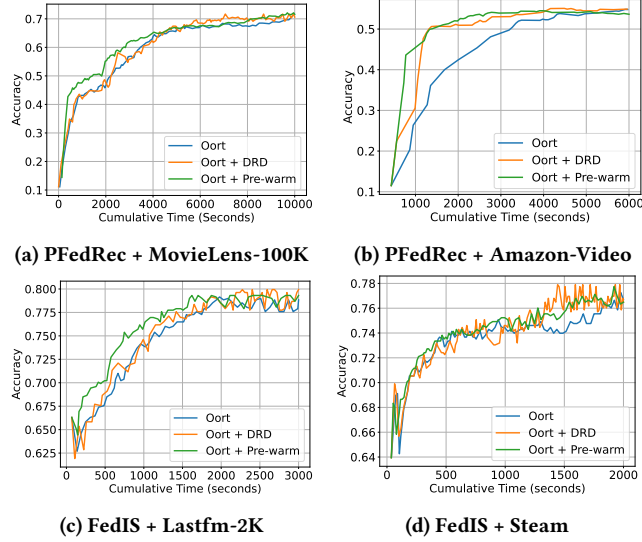


Figure 13: Model Accuracy Change During Training Window

**Model Convergence Time** For this metric, we observed that:

- 1) Phone-based FRS achieved the fastest model convergence among all approaches, as phones typically have more computational resources for local training than routers. However, the model accuracy quickly plateaued at a low level. This is because, within the short training window, no new phones became available to contribute additional user data. As a result, the model was trained on a fixed set of users, leading to rapid convergence but low final accuracy.
  - 2) For router-based FRS, our strategy consistently reduced the time required to reach model convergence compared to both Oort and FLOAT across all four cases. For example, compared to Oort, our strategy reduced training time by 40% for *PFedRec + MovieLens-100K*, 43% for *PFedRec + Amazon-Video*, 23% for *FedIS + Lastfm-2K*, and 42% for *FedIS + Steam*. On average, our approach achieved a 37% reduction in model convergence time across these cases. Importantly, this improvement in training efficiency is achieved without sacrificing final model accuracy. These results indicate that, even when real-world apps operate under tighter training time windows, our strategy enables faster convergence and delivers more accurate recommendations compared to state-of-the-art baselines.
- Ablation Study** We then conducted an ablation study to evaluate the contribution of each mechanism in our proposed strategy: (i) *Dynamic Round Deadline (DRD)* dynamically determines a deadline for each round to select routers that maximize overall utility, thereby accelerating model convergence; and (ii) *Pre-warm* performs local training on each router prior to the online training window to ensure that every user's data is incorporated at least once. Since both mechanisms are compatible with existing straggler-mitigation

strategies such as Oort and FLOAT, we use Oort as the representative baseline and separately add each component to observe their individual impact on performance.



**Figure 14: Breakdown of Model Convergence Performance**

Figure 14 presents the model convergence time across all cases when the two mechanisms are applied individually. We observed:

- 1) Both *DRD* and *Pre-warm* mechanisms played a non-trivial role in reducing the time required for model convergence.

- 2) *Pre-warm* primarily enhances performance in the early stages of training. This is because, beyond updating model parameters through offline local training, it also records utility profiling data, which the selection algorithm can immediately leverage for utility-based selection. In contrast, existing strategies such as Oort and FLOAT suffer from cold-start delays, as they rely on random selection during the initial rounds to gather utility profiles before switching to utility-based selection. While such delays may be negligible in other FL tasks like computer vision, which often have training windows spanning days, they can introduce substantial latency in recommendation tasks with limited training windows. *Pre-warm* effectively eliminates this delay.

- 3) While *Pre-warm* primarily improves early training performance, *DRD* provides consistent benefits throughout the whole training process by refining router selection in every round based on updated training times and utility scores.

## 5.2 Evaluation of Middleware Runtime

**5.2.1 Experimental Methodology.** We introduce how we implement our middleware and configure the experiments to compared with other existing recommendation approaches.

**System Implementation** Our testbed consists of an Android phone (Samsung Galaxy A53 with Octa-core CPUs and 6GB RAM) as the mobile client, and a Raspberry Pi and PC used in our emulation platform, serving as the home router and cloud server, respectively. Both the phone and Raspberry Pi are connected to the same local network, while the PC is placed 6 network hops away to simulate remote server access.

For the client, we modified *OkHttp* (version 3.14.9) [40], an open-source Android HTTP client library, to handle interactions between

the app and the router and the cloud. Specifically, the library retrieves the IP address of the client’s DNS as the router’s IP address, considering that home routers typically act as local DNS servers for associated devices. For the router runtime, we set up a *Flask* [44] HTTP server to handle requests from the app. When the router receives a request from the *Start Page*, it invokes the local model inference module (implemented by *PyTorch* (version 2.5.0) [13]) to generate recommended items, which are then returned to the app.

**Evaluated App that Uses Cloud-based RS** To compare how FRS would perform compared with cloud-based RS, we selected the model trained with *PFedRec+MovieLens-100k* and deployed all its model parameters (i.e., the private embeddings of all users, public parameters on the cloud server as a cloud-based RS. The server’s role consists of: 1) Generating Recommendation: Upon receiving a request from an app user, it retrieve the user’s corresponding private embeddings and runs model inference to generate IDs of top ranked movies. 2) Serving Data: movie details (i.e., cast description, video link). We then developed an app to use this cloud-based RS. Specifically, the app’s *Start Page* sends a HTTP request to fetch 50 recommended movies’ ID, and the *Item Page* is to fetch the details of the movie clicked by the user.

**Evaluated Approaches** We evaluated the following approaches of integrating the recommendation functionality of FRS into the above app that uses cloud-based RS.

- 1) *Phone-based FRS*: We modified the app to integrate on-device recommendation into the app. We used *LiteRT* (version 2.7)[5] to implement the local model training and inference in the app.
- 2) *Router-based FRS + Automatic Integration*: This is our middleware solution. We used the programming model to add the annotations to handle the interactions between the app and the router or the app and the cloud server. This enabled the recommendation functionality to be automatically integrated into the app.
- 3) *Router-based FRS + Manual Integration*: Instead of using our programming model to integrate the recommendation, we manually modified the app’s source code to handle the interactions. This involved tasks such as retrieving the router’s IP address and issuing separate HTTP requests to the router and the cloud server.

**5.2.2 Performance Comparison.** We compare performance of these approaches for usability and runtime latency.

**Usability Comparison** We compared the usability of these approaches through both quantitative and qualitative measures. Quantitatively, we assessed the number of lines of code (LoC) required and the additional size of the app installation package. Qualitatively, we evaluated ease of use by examining whether app developers need to modify the app logic.

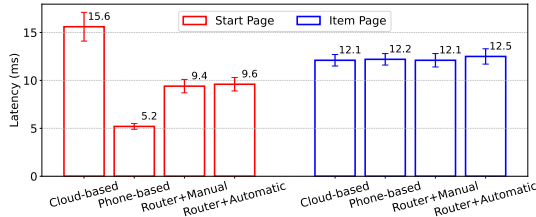
| Approach           | LoC | Extra App Size (KB) | Re-write Logic? |
|--------------------|-----|---------------------|-----------------|
| Phone-based FRS    | 982 | 76,000              | Yes             |
| Manual Integration | 210 | 54                  | Yes             |
| Our Middleware     | 2   | 54                  | No              |

**Table 3: Comparison of Usability**

As shown in Table 3, we observed that 1) *Phone-based FRS* required developers to write extensive code to modify the app logic, and resulted in the largest increase in app size due to the need to packaging the recommendation model parameters directly in the app; 2) Our middleware, in contrast, only required developers to add a few Java annotations without necessitating any modifications to the app logic, setting it apart from the other two baselines.

Modifying the app logic not only demands additional coding effort but also increases the risk of introducing bugs, which may require extra debugging and testing time to resolve. Regarding the extra app size, both our approach and *manual integration* introduced the smallest increase in size, as model parameters were stored on the routers rather than within the app.

**Recommendation Serving Latency** We evaluated the recommendation serving latency by measuring the end-to-end latency of HTTP requests for both the *Start Page* and *Item Page*. Specifically, we tested three versions of the app implemented by the three approaches, along with the version utilizing cloud-based RS. During each test, we used *Auto Clicker* [46], an Android tool for automating screen touches, to first launch the *Start Page* and then click the first returned item to retrieve data for the *Item Page*. Each test ran for 1 hour, with a 20-second interval between touches.



**Figure 15: Recommendation Serving Latency Comparison**

From Figure 15, we observed: 1) For the *Start Page*, the router-based FRS exhibited slightly higher latency compared to phone-based FRS because the latter performs model inference locally without the need to communicate with a router. However, the router-based FRS outperformed the cloud-based RS by avoiding the additional network latency associated with contacting a remote server. 2) For the *Item Page*, all approaches demonstrated similar latency performance as it fetches the data from the cloud. This also highlighted that our programming model introduced negligible latency overhead (comparing case *Router+Manual* with case *Router+Automatic*), despite the additional processes of interception and redirection within the HTTP client library.

## 6 Related Work and Discussion

We discuss the existing studies on FRS, federated training strategies, and the data security in this section.

**Federated Recommendation Systems** FRS have attracted much attention recently for resolving the privacy issues in cloud-based RS. Many studies have focused on developing advanced algorithms for FRS. For instance, FCF [4] is the first FRS based on matrix factorization (MF), utilizing stochastic gradient descent for local model updates and FedAvg for global model aggregation. Further advancements [36, 42, 56] integrate additional deep neural networks with MF to capture the complex non-linear relationships between users and items. To prevent privacy threats of reverse-engineering from model gradients, FedMF [10] adds homomorphic encryption to secure gradients before they are uploaded to the cloud server.

Despite these extensive theoretical advancements, the practical deployment of FRS remains limited. This is mainly due to the reliance on mobile phones as training clients, causing compromised model accuracy due to the low availability of these devices and significant programming overheads on developers. Motivated by the consistent computational and communication resources provided

by personal home routers, which are available 24/7, we propose leveraging these devices as training clients for FRS. This approach aims to achieve high recommendation accuracy, privacy preservation and high usability simultaneously.

**Stragglers Optimization in Federated Training** Federated training takes place in heterogeneous environments, where clients vary significantly in both data distribution and hardware capabilities. To address the straggler problem—which can delay model convergence and degrade accuracy—numerous approaches have been proposed. Oort [30] introduces a client selection strategy that balances the trade-off between data utility and system efficiency to optimize time-to-accuracy. Building on this core idea, subsequent works have introduced specific optimizations: PyramidFL [33] and FLOAT [27] allow clients to train for different numbers of local epochs based on their capacity, while others such as Fjord [25] and HeteroFL [17] adaptively update only subsets of model weights depending on device capabilities. The mechanisms proposed in this work—namely, pre-warm and dynamic round deadline—are implemented and evaluated on top of Oort and FLOAT. However, they are generalizable and can be incorporated into other federated training frameworks to further enhance the training efficiency and model accuracy in router-based FRS.

**Data Security** As the gateway of a home network, the home router manages all data flow between the private local network and the public Internet. Its critical role makes it a prime target for various attacks, including botnet malware infections [24], exploits of wireless protocol implementations [49], and privilege escalation attacks [19], all of which pose significant risks to user data and privacy. This security concern has been widely acknowledged by the computer security community, leading to the development of mitigation and defense strategies. For example, some research efforts have proposed new router operating systems that isolate application data processing within sandboxes, thereby preventing data leakage even if the router is compromised [31, 32]. Beyond sandboxing, other mechanisms such as encrypting user data locally and erasing it after each training window can be applied to further enhance the user data security.

## 7 Conclusion

This paper presents a novel federated recommendation system architecture that leverages personal home routers to deliver accurate, timely and privacy-preserving recommendations. We develop a lightweight middleware system featuring a programming model for easy app integration and propose a customized training strategy to address router straggler issues. Extensive evaluation demonstrates that our approach improves recommendation accuracy by up to 52% compared to traditional phone-based FRS, reduces convergence time by up to 43% relative to state-of-the-art training strategies, and imposes minimal programming and runtime overheads.

## Acknowledgement

We sincerely thank the anonymous reviewers and our shepherd Lydia Chen for their insightful feedback. This research is supported by NSF through the grants #2104337.



## REFERENCES

- [1] Ahmed M Abdelmoniem, Atal Narayan Sahu, Marco Canini, and Suhaib A Fahmy. 2023. Refl: Resource-efficient federated learning. In *Proceedings of the eighteenth European conference on computer systems*. 215–232.
- [2] Accessible Law. 2021. Personal Information Protection Law of the People's Republic of China. <https://personalinformationprotectionlaw.com/> An unofficial English translation and web version of the PIPL.
- [3] Marie Al-Ghossein, Talel Abdessalem, and Anthony Barré. 2021. A survey on stream-based recommender systems. *ACM computing surveys (CSUR)* 54, 5 (2021), 1–36.
- [4] Muhammad Ammad-Ud-Din, Elena Ivannikova, Suleiman A Khan, Were Oyomno, Qiang Fu, Kuan Eeik Tan, and Adrian Flanagan. 2019. Federated collaborative filtering for privacy-preserving personalized recommendation system. *arXiv preprint arXiv:1901.09888* (2019).
- [5] TensorFlow Authors. 2021. TensorFlow Lite. <https://www.tensorflow.org/lite> Available at <https://www.tensorflow.org/lite>.
- [6] Nathaniel Bastian, David Bierbrauer, Morgan McKenzie, and Emily Nack. 2023. ACI IoT Network Traffic Dataset 2023. doi:10.21227/qacj-3x32
- [7] Jesús Bobadilla, Fernando Ortega, Antonio Hernandez, and Abraham Gutiérrez. 2013. Recommender systems survey. *Knowledge-based systems* 46 (2013), 109–132.
- [8] Keith Bonawitz. 2019. Towards federated learning at scale: System design. *arXiv preprint arXiv:1902.01046* (2019).
- [9] Iván Cantador, Peter Brusilovskiy, and Tsvi Kuflik. 2011. Second workshop on information heterogeneity and fusion in recommender systems (HetRec2011). In *Proceedings of the fifth ACM conference on Recommender systems*. 387–388.
- [10] Di Chai, Leye Wang, Kai Chen, and Qiang Yang. 2020. Secure federated matrix factorization. *IEEE Intelligent Systems* 36, 5 (2020), 11–20.
- [11] Hyunsung Cho, Akhil Mathur, and Fahim Kawsar. 2021. Device or user: Re-thinking federated learning in personal-scale multi-device environments. In *Proceedings of the 19th ACM Conference on Embedded Networked Sensor Systems*. 446–452.
- [12] OpenWRT Community. 2025. OpenWRT: Linux operating system for embedded devices. <https://openwrt.org/> Available at <https://openwrt.org/>.
- [13] PyTorch Contributors. 2025. PyTorch: An open source machine learning framework. <https://pytorch.org> Available at <https://pytorch.org>.
- [14] Wikipedia Contributors. 2025. Tomato (firmware). [https://en.wikipedia.org/wiki/Tomato\\_\(firmware\)](https://en.wikipedia.org/wiki/Tomato_(firmware)) Available at [https://en.wikipedia.org/wiki/Tomato\\_\(firmware\)](https://en.wikipedia.org/wiki/Tomato_(firmware)).
- [15] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman. 2005. *Linux Device Drivers* (3rd ed.). O'Reilly Media. <https://lwn.net/Kernel/LDD3/> Chapter 1 and 2 discuss kernel vs. user space and the ability of kernel code to preempt user-space execution.
- [16] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [17] Enmao Diao, Jie Ding, and Vahid Tarokh. 2020. Heterofl: Computation and communication efficient federated learning for heterogeneous clients. *arXiv preprint arXiv:2010.01264* (2020).
- [18] Xuanang Ding, Guohui Li, Ling Yuan, Lu Zhang, and Qian Rong. 2023. Efficient federated item similarity model for privacy-preserving recommendation. *Information Processing & Management* 60, 5 (2023), 103470.
- [19] Mohamed Elsbagh, Ryan Johnson, Angelos Stavrou, Chaoshun Zuo, Qingchuan Zhao, and Zhiqiang Lin. 2020. {FIRMSCOPE}: Automatic uncovering of {Privilege-Escalation} vulnerabilities in {Pre-Installed} apps in android firmware. In *29th USENIX security symposium (USENIX Security 20)*. 2379–2396.
- [20] Ido Guy, Naama Zwerdling, Inbal Ronen, David Carmel, and Erel Uziel. 2010. Social media recommendation based on people and tags. In *Proceedings of the 33rd international ACM SIGIR conference on Research and development in information retrieval*. 194–201.
- [21] F Maxwell Harper and Joseph A Konstan. 2015. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)* 5, 4 (2015), 1–19.
- [22] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*. 173–182.
- [23] Xinrui He, Shuo Liu, Jacky Keung, and Jingrui He. 2024. Co-clustering for federated recommender system. In *Proceedings of the ACM Web Conference 2024*. 3821–3832.
- [24] Stephen Hilt and Fernando Mercés. 2021. Vpnfilter two years later: Routers still compromised.
- [25] Samuel Horvath, Stefanos Laskaridis, Mario Almeida, Ilias Leontiadis, Stylianos Venieris, and Nicholas Lane. 2021. Fjord: Fair and accurate federated learning under heterogeneous targets with ordered dropout. *Advances in Neural Information Processing Systems* 34 (2021), 12876–12889.
- [26] Hyunwoo Hwangbo, Yang Sok Kim, and Kyung Jin Cha. 2018. Recommendation system development for fashion retail e-commerce. *Electronic Commerce Research and Applications* 28 (2018), 94–101.
- [27] Ahmad Faraz Khan, Azal Ahmad Khan, Ahmed M Abdelmoniem, Samuel Fountain, Ali R Butt, and Ali Anwar. 2024. Float: Federated learning optimizations with automated tuning. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 200–218.
- [28] Colin Ian King. 2014. stress-ng: Stress Test and Benchmark Tool. <https://manpages.ubuntu.com/manpages/focal/man1/stress-ng.1.html> Accessed: 2024-12-29.
- [29] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [30] Fan Lai, Xiangfeng Zhu, Harsha V Madhyastha, and Mosharaf Chowdhury. 2021. Oort: Efficient federated learning via guided participant selection. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*. 19–35.
- [31] Seung-seob Lee, Hang Shi, Kun Tan, Yunxin Liu, SuKyoung Lee, and Yong Cui. 2016. Smart and secure: Preserving privacy in untrusted home routers. In *Proceedings of the 7th ACM SIGOPS Asia-Pacific Workshop on Systems*. 1–8.
- [32] Seung-seob Lee, Hang Shi, Kun Tan, Yunxin Liu, SuKyoung Lee, and Yong Cui. 2019. S2Net: Preserving privacy in smart home routers. *IEEE Transactions on Dependable and Secure Computing* 18, 3 (2019), 1409–1424.
- [33] Chenning Li, Xiao Zeng, Mi Zhang, and Zhichao Cao. 2022. PyramidFL: A fine-grained client selection framework for efficient federated learning. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*. 158–171.
- [34] Yue Li, Tian Tan, and Jingling Xue. 2019. Understanding and analyzing java reflection. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 2 (2019), 1–50.
- [35] Feng Liang, WeiKe Pan, and Zhong Ming. 2021. Fedrec++: Lossless federated recommendation with explicit feedback. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 4224–4231.
- [36] Zhaohao Lin, WeiKe Pan, Qiang Yang, and Zhong Ming. 2022. A generic federated recommendation framework via fake marks and secret sharing. *ACM Transactions on Information Systems* 41, 2 (2022), 1–37.
- [37] Rachana Mehta and Keyur Rana. 2017. A review on matrix factorization techniques in recommender systems. In *2017 2nd International Conference on Communication Systems, Computing and IT Applications (CSCITA)*. IEEE, 269–274.
- [38] Gilad Mishne, Jeff Dalton, Zhenghua Li, Aneesh Sharma, and Jimmy Lin. 2013. Fast data in the era of big data: Twitter's real-time related query suggestion architecture. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 1147–1158.
- [39] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 188–197.
- [40] okhttp. [n. d.]. <https://square.github.io/okhttp/>.
- [41] European Parliament and Council. 2016. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data (General Data Protection Regulation). Retrieved 9 20, 2023 from <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [42] Vasileios Perifanis and Pavlos S Efraimidis. 2022. Federated neural collaborative filtering. *Knowledge-Based Systems* 242 (2022), 108441.
- [43] Laércio Lima Pilla. 2021. Optimal task assignment for heterogeneous federated learning devices. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 661–670.
- [44] Pallets Projects. 2025. Flask Documentation (Stable). <https://flask.palletsprojects.com/en/stable/> Available at <https://flask.palletsprojects.com/en/stable/>.
- [45] Ricardo Batista Rodrigues, CM da Silva, Wilton O Ferreira, and Glauca MM. 2013. A Cloud-based recommendation system. In *International conference on WWW/Internet*. 384–386.
- [46] Softonic. 2025. Auto Clicker: Automatic Touch. <https://auto-clicker-automatic-touch.en.softonic.com/android> Available at <https://auto-clicker-automatic-touch.en.softonic.com/android>.
- [47] State of California. 2018. California Consumer Privacy Act of 2018 (CCPA). <https://oag.ca.gov/privacy/ccpa> Accessed: 2025-01-02.
- [48] Tamber. 2017. Steam Video Games. <https://www.kaggle.com/datasets/tamber/steam-video-games> Accessed: 2024-12-31.
- [49] Mathy Vanhoef and Frank Piessens. 2018. Release the Kraken: new KRACKs in the 802.11 Standard. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 299–314.
- [50] Cong Wang, Xin Wei, and Pengzhan Zhou. 2020. Optimize scheduling of federated learning on battery-powered mobile devices. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 212–221.
- [51] Qinyong Wang, Hongzhi Yin, Tong Chen, Junliang Yu, Alexander Zhou, and Xiangliang Zhang. 2022. Fast-adapting and privacy-preserving federated recommender system. *The VLDB Journal* 31, 5 (2022), 877–896.

- [52] Wikipedia contributors. 2024. JSON – Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/wiki/JSON> [Online; accessed 9-February-2025].
- [53] Chengxu Yang, Qipeng Wang, Mengwei Xu, Zhenpeng Chen, Kaigui Bian, Yunxin Liu, and Xuanzhe Liu. 2021. Characterizing impacts of heterogeneity in federated learning upon large-scale smartphone data. In *Proceedings of the Web Conference 2021*. 935–946.
- [54] Timothy Yang, Galen Andrew, Hubert Eichner, Haicheng Sun, Wei Li, Nicholas Kong, Daniel Ramage, and Françoise Beaufays. 2018. Applied federated learning: Improving google keyboard query suggestions. *arXiv preprint arXiv:1812.02903* (2018).
- [55] Wei Yuan, Liang Qu, Lizhen Cui, Yongxin Tong, Xiaofang Zhou, and Hongzhi Yin. 2024. Hetefedrec: Federated recommender systems with model heterogeneity. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1324–1337.
- [56] Chunxu Zhang, Guodong Long, Tianyi Zhou, Peng Yan, Zijian Zhang, Chengqi Zhang, and Bo Yang. 2023. Dual personalization on federated recommendation. *arXiv preprint arXiv:2301.08143* (2023).
- [57] Chen Zhang, Yu Xie, Hang Bai, Bin Yu, Weihong Li, and Yuan Gao. 2021. A survey on federated learning. *Knowledge-Based Systems* 216 (2021), 106775.